

# Usage-Centered Engineering for Web Applications

Larry L. Constantine and Lucy A.D. Lockwood, *Constantine & Lockwood*

**D**espite breathless declarations that the Web represents a new paradigm defined by new rules, professional developers are realizing that lessons learned in the pre-Internet days of software development still apply. Web pages are user interfaces, HTML programming is programming, and browser-deployed applications are software systems that can benefit from basic software engineering principles.

Nevertheless, development for the Web faces difficulties that standard development does not. Three challenges come to the fore. First, the Web development process must be highly accelerated—"Web time" has become synonymous with headlong desperation and virtually impossible deadlines. Second, many Web projects are "green-field" applications, which means they have few if any standards or precedents. Finally, much more so than standard software, Web applications must focus on the user experience.

Systematic, disciplined practices might help developers face these challenges, but traditional approaches proliferate documents and deliverables and emphasize analysis of established practices—a poor match for novel applications with Web-time delivery schedules. Furthermore, user interface design and usability are generally weak points in both the heavyweight software en-

gineering processes, such as the Unified Process,<sup>1</sup> and the lightweight competitors, such as Extreme Programming.<sup>2</sup> This article describes a flexible, model-driven approach for engineering Web applications that succeeds through a focus on user interface design and usability. Its simple, model-driven techniques work well for novel applications and integrate readily with *lightweight* or *agile* development processes<sup>3</sup> under compressed development schedules.

## Web applications for use

Usability and the user experience are emerging as critical determinants of success in Web applications. If customers can't find what they're looking for, they can't buy it; a site that buries key information impairs business decision making. Poorly designed interfaces increase user errors, which can be costly. Mistakes made entering credit card billing information, for example, can re-

**This model-driven approach to software engineering, focusing on user interface design and usability, proves especially appropriate for Web applications.**

**Table 1****User-centered and usage-centered design:  
A comparison**

| User-centered design                                                                                                                              | Usage-centered design                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Focus is on users: User experience and user satisfaction                                                                                          | Focus is on usage: Improved tools supporting task accomplishment                                                                      |
| Driven by user input                                                                                                                              | Driven by models and modeling                                                                                                         |
| Substantial user involvement                                                                                                                      | Selective user involvement                                                                                                            |
| <ul style="list-style-type: none"> <li>• User studies</li> <li>• Participatory design</li> <li>• User feedback</li> <li>• User testing</li> </ul> | <ul style="list-style-type: none"> <li>• Exploratory modeling</li> <li>• Model validation</li> <li>• Usability inspections</li> </ul> |
| Design by iterative prototyping                                                                                                                   | Design by modeling                                                                                                                    |
| Highly varied, informal, or unspecified processes                                                                                                 | Systematic, fully specified process                                                                                                   |
| Design by trial and error, evolution                                                                                                              | Design by engineering                                                                                                                 |

quire costly manual follow-up or lead to lost sales. Web-usability guru Jakob Nielsen estimates that billions of dollars in lost Web sales can be traced to usability problems.

Technical and customer support costs in all areas of business are skyrocketing. Every time usability problems on a site or application prompt a telephone call or an email message to customer service, an inexpensive Web session becomes an expensive support incident. On corporate intranets, hard-to-use applications require extensive training or go completely unused.

Although this article focuses on Web-based applications—systems that deploy functional capability on the Web through a browser instance as a thin client—it is difficult to draw a hard and fast line separating Web-based applications from those Web sites that are intended for use, that is, for something more than entertainment, “branding,” or the like. The common thread is the delivery of useful services and capabilities over the Web.

In addition to usability, aesthetic design aspects also figure in the user experience. Unfortunately, graphic design and aesthetic considerations are often established early and drive the entire development process at the expense of usability. For example, based on an “aesthetic brief” approved by a client’s CEO, a prominent graphic design house developed an innovative Web site for a major analyst group. Although the designers consulted extensively with the firm’s customers and conducted numerous market-oriented focus groups, the design proved a usability disaster, in no small part because of the aesthetic design. Last-ditch efforts to make improvements without modifying the basic graphic design enabled the site to go live but ultimately proved merely palliative and insufficient to satisfy customers. In the end, the firm redesigned the site completely, this time approaching the project with usage-centered design.<sup>4</sup>

### Usage-centered design

Usage-centered design uses abstract models to systematically design the smallest, simplest system that fully and directly supports all the tasks users need to accomplish. Developed in the early 1990s, the approach has been applied to systems ranging from industrial automation systems and consumer elec-

tronics to banking and insurance applications. The streamlined process, driven by simple models, scales readily; developers have used it on projects ranging from a few person-months to the five-designer, 19-developer, 23-month project that produced Step7lite, a sophisticated integrated development environment from Siemens AG (see <http://foruse.com/pcd>). On the Web, groups have employed usage-centered design successfully for applications in fields including e-commerce, membership support, education, and medical informatics.<sup>5</sup>

### User-centered or usage-centered?

Usage-centered design evolved as a software engineering alternative to user-centered design. Table 1 summarizes some of the salient differences. User-centered design is a loose collection of human-factors techniques united under a philosophy of understanding users and involving them in design. It relies primarily on three techniques: user studies to identify users’ desires, rapid paper prototyping for user feedback on user interface design iterations, and usability testing to identify problems in working prototypes or systems. Although helpful, none of these techniques can replace good design. User studies can easily confuse what users want with what they truly need. Rapid iterative prototyping can often be a sloppy substitute for thoughtful and systematic design. Most importantly, usability testing is a relatively inefficient way to find problems you can avoid through proper design.<sup>4,6</sup>

**Driving models.** *Role, task, and content* models—simple, closely related abstract mod-

### Withdraw Money

The use case begins when the client inserts an ATM card. The system reads and validates the information on the card.

1. System prompts for PIN. The client enters PIN. The system validates the PIN.
2. System asks which operation the client wishes to perform. Client selects “Cash withdrawal.”
3. System requests amounts [*sic*]. Client enters amount.
4. System requests type. Client selects account type (checking, savings, credit).
5. The system communicates with the ATM network to validate account ID, PIN, and availability of the amount requested.
6. The system asks the client whether he or she wants a receipt. This step is performed only if there is paper left to print the receipt.
7. System asks the client to withdraw the card. Client withdraws card. (This is a security measure to ensure that Clients do not leave their cards in the machine.)
8. System dispenses the requested amount of cash.
9. System prints receipt.
10. The use case ends.

**Figure 1. Conventional use case for getting cash from an ATM.<sup>9</sup>**

| <u>Getting cash</u> |                         |
|---------------------|-------------------------|
| User intentions     | System responsibilities |
|                     | 1. Request identity.    |
| 2. Identify myself. | 3. Verify identity.     |
|                     | 4. Offer choices.       |
| 5. Make choice.     | 6. Give requested cash. |
| 7. Take cash.       |                         |
|                     |                         |

**Figure 2. Abstract use case in essential form—the “getting cash” task case.**

els—drive usage-centered design. The role model captures the salient characteristics of the roles that users play in relation to a system. The task model represents the structure of the work users need to accomplish with the system. The content model represents the user interface’s contents and organization.

A user role represents one kind of relationship users could have with a system. Among numerous possible aspects of this relationship are the purpose and frequency of interaction, the volume and direction of information exchange, and the attitude toward the system of users in the role.

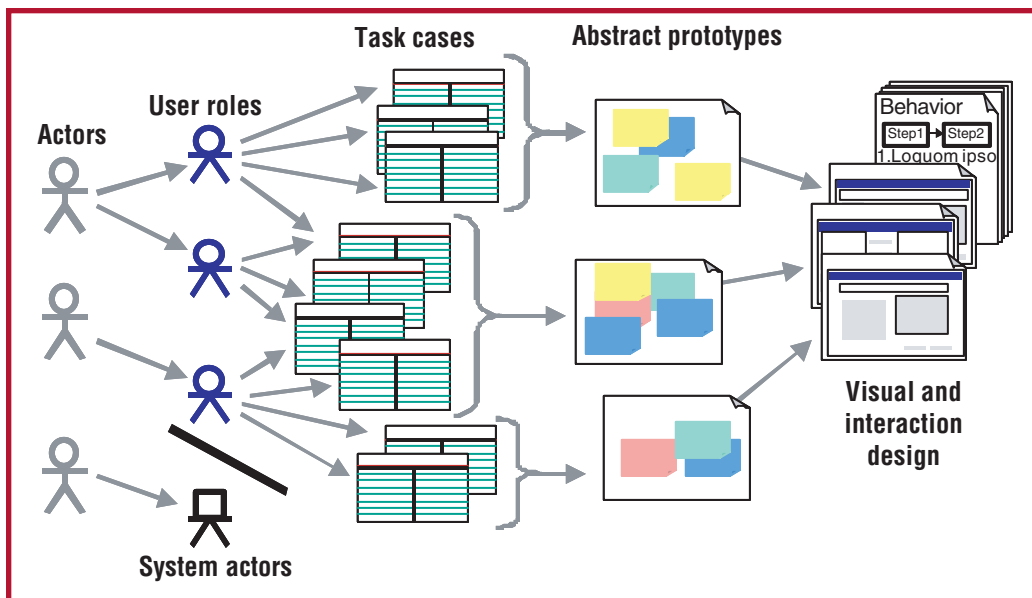
A task model comprises a set of task cases

and a map of the interrelationships among those task cases. Task cases are a form of use cases,<sup>7</sup> which conventionally are models of systems—namely, the “sequence of actions a system performs that yields an observable result to a particular actor.”<sup>1</sup> As commonly written, use cases express the concrete actions and responses taken by the system and an actor with which it interacts. Figure 1, a published example of a conventional use case,<sup>1</sup> tellingly omits the user’s single most important step: actually taking the cash.

Task cases—also called essential use cases<sup>4</sup>—are essential models<sup>8</sup> that are abstract, simplified, and technology-free—that is, they have no built-in assumptions about the user interface. For example, the task case of Figure 2, written in the style used in agile usage-centered design, is shorter, simpler, and closer to the essence of the user task than that of Figure 1. The advantages of essential use cases are widely recognized, and a consensus is building that this style is best for both user interface design and requirements modeling in general.<sup>9</sup>

The third model, the content model,<sup>4</sup> is sometimes called an abstract prototype: It abstractly represents the user interface’s contents—independent of actual appearance or behavior—and how these are organized into the contexts within which users interact, called *interaction contexts*. Abstract prototypes can range from a simple inventory of a site’s or application’s accessible contents to a highly structured abstract model based on a canonical set of abstract components (see <http://foruse.com/articles/canonical.pdf>). Regardless of the form, abstract prototypes are a bridge between task models and realistic or representational prototypes, such as conventional paper prototypes or design sketches. By adhering to a user perspective and leaving open the various implementation possibilities, abstraction encourages innovative visual and interaction designs.<sup>4,10</sup>

**Process overview.** Figure 3 shows a logical overview of usage-centered design. The process separates system actors—other software and hardware systems—from human actors who are users. The role, task, and content models are developed in coordination with and interlinked with other models. These include a domain model (such as a



**Figure 3. Logical process of usage-centered design.**

glossary or another more elaborate data model), a business rules model that embodies the application's underlying logic and constraints,<sup>11</sup> and an operational model that captures salient aspects of the working environment or operational context.<sup>3</sup>

Conceptually, a straightforward and direct derivation links the final design back to task cases supporting user roles. Each page, form, or other interaction context corresponds to an abstract prototype supporting a cluster of interrelated task cases. The designer derives the actual buttons, links, tables, displays, and other features directly from abstract components that realize specific steps within supported task cases. These task cases, in turn, support the roles that users can perform.

### Usage-centered Web design

Over successive Web projects, usage-centered design has evolved a lightweight form that takes certain modeling and design shortcuts (see <http://foruse.com/articles/agiledesign.pdf>). Agile usage-centered design employs rapid-fire modeling techniques using ordinary index cards for brainstorming, sorting, and clustering,<sup>12</sup> as in Extreme Programming.<sup>2</sup> When practical, collaborative modeling with end users, application domain experts, and clients can save additional time. Figure 4 outlines an agile usage-centered design process.

**Essential purpose and preconception.** Web projects often suffer from vague objectives and unrealistic ambitions. Early clarification of objectives and exploration of fantasies can pay off enormously.<sup>13</sup> We prefer

to kick off the process by involving managers, users, developers, and other stakeholders in framing the application's purposes as external users and from a business perspective. The group can brainstorm onto cards and sort them to establish priorities. Similarly, we encourage participants to share their fantasies about the system's features, functions, facilities, content, and capabilities, but we clearly label these ideas preconceptions and set them aside to be treated as negotiable fantasies rather than requirements.

**Exploratory modeling.** Exploratory modeling jumpstarts the design process by identifying questions and areas of uncertainty or ambiguity in user requirements. The team does this by sketching out rough versions of the role and task models that it will develop in more refined form later. In this way, the team can identify needed, but missing, information.

**User interface architecture.** Many agile processes<sup>3</sup> abjure up-front design in favor of a more evolutionary approach—constructing a limited system and then elaborating it through successive iterations. Unfortunately, when code starts small and gradually grows with expanding requirements, the original architecture—the code's underlying organization—often proves insufficient or inappropriate to support emerging needs. *Refactoring*,<sup>14</sup> in which existing code is reorganized and rewritten to fit evolving and emerging requirements, is a key to successful evolution of complex software systems through iterative expansion.

### Preliminary steps

1. *Essential purpose and preconception*—Clarify business and user purposes, then fantasize and set aside preconceptions of features, facilities, content, and capabilities.
2. *Exploratory modeling*—Identify questions, ambiguities, and areas of risk and uncertainty.

### First iteration

3. *Role modeling*—Inventory and prioritize all user roles and select initially targeted subset.
4. *Task modeling*—Inventory and prioritize all tasks and select initially targeted subset.
5. *Task clustering*—Group all tasks by affinity and draft overall navigation architecture.
6. *Design*—Draft visual and interaction scheme; review and revise aesthetic design.
7. *Abstract prototyping*—Develop content model for interaction contexts supporting selected subset of tasks.
8. *Design*—Develop detailed user interface design for selected interaction contexts.
9. *Construction*—Program designed portions of user interface.

### Successive iterations (similarly)

- 3a. Review role model and revise as needed, select next subset.
- 4a. Review task model and revise as needed, select next subset.
- 5a. Review navigation architecture and revise as needed.
- 6a. Review visual and interaction scheme and revise as needed.
- 7a. Develop content model for interaction contexts supporting next selected task subset.
- 8a. Design user interface for selected interaction contexts.
- 9a. Construct newly designed portions.

**Figure 4. Rough outline of steps in an agile usage-centered design process.**

User interfaces are a different story. Late refinement of the user interface's basic structure is not acceptable because it changes the system for users who have already learned or mastered an earlier version. Whereas refactored internal software components don't necessarily affect users, a redesigned user interface architecture is unavoidably disruptive. For highly usable user interfaces, we design the overall organization, navigation, and look and feel to fit the full panoply of tasks to be covered. You don't have to design every aspect of the user interface in advance, however. To avoid radical refactoring or inconsistency across design increments, you need to work out a well-conceived overall map of the site or application in advance.<sup>15</sup> To insure that the basic visual style and be-

havior are effective across the entire application, you must devise a comprehensive design scheme.

Navigation architecture specifies how the overall user interface breaks down into interaction contexts, collections, and groups; how these are presented to users; and how users navigate them. Minimally, the navigation architecture identifies all the interaction contexts and their interconnections, but having more is advantageous. For example, the navigation architecture might organize parts of the user interface into tabbed notebooks—accessed through a set of command buttons on a central dispatch dialog—with sections within notebook pages reached through shortcuts in a menu-like bar across the top of each page.

Another crucial facet of overall user interface design is the visual and interaction scheme, a sort of abstract style guide. It briefly describes the basic visual elements that will recur throughout the design as well as the common layouts, visual arrangements, or templates that apply to various interaction contexts.<sup>16</sup> The scheme might, for example, specify a basic layout grid with a modified tree-view in the left-most column for primary navigation and a set of view controls across the top for secondary navigation. At a more detailed level, it might specify that a distinct color should identify all editable fields, which can be edited in place by double clicking. It might further spell out how certain collections of controls will be placed on slide-out tool panels that automatically open on mouse-over and close after use.

Clearly, a sound navigation architecture and visual and interaction scheme requires a complete task model that includes all identified tasks, not just those to be tackled on the first or early iterations.

**Card-based modeling.** The initial user role model is a simple inventory of roles users can play in relation to the site or application. One of the quickest ways to develop such an inventory is to brainstorm it directly onto index cards. Once we've agreed on the initial inventory, we succinctly describe roles directly on their index cards using the characteristics most relevant to the design problem: the role's context, characteristic patterns of interaction within the

role, and special support criteria. We then sort the completed cards to rank them by overall importance.

After reviewing the roles in order of importance, we brainstorm an inventory of task cases, again onto index cards. As we review and refine this inventory—condensing, combining, eliminating, or adding tasks—we standardize the names to convey the basic purpose from a user’s perspective, such as “finding a replacement part for a specific product” or “adding an assessment to a lesson plan.” Once we’ve established the initial set of task cases, we sort the cards to rank them for expected frequency and for overall importance. Based on these two rankings, we deal them into three piles: required (do first), desired (do if there’s time), and deferred (do next time).

At this point, we start to fill in the details, but not for all task cases. We skip task cases for which the interaction appears obvious or routine. For task cases that are critical, complex, unclear, or interesting, we write the interaction narrative on the index card. Working on small cards favors good modeling discipline. If the process narrative doesn’t fit on one card, either the narrative is not sufficiently abstract and simplified, or the card really covers multiple task cases that we must identify.

Using all the knowledge built from creating, refining, and interacting with the task cases, we group the cards into affinity clusters on the basis of how strongly they seem to be related—particularly, how likely users in a role will perform tasks together. Each cluster represents a set of capabilities that the application must offer together—within the same page, form, or browser window, or at least within a set of closely connected interaction contexts.

Each task cluster becomes the guide for designing a part of the user interface. We prefer to construct abstract prototypes but under pressure could move directly into sketching realistic paper prototypes. We then evaluate the paper prototypes, augmented by a description of how the various elements behave, to identify usability problems and areas for improvement. For speed and convenience, we prefer collaborative usability inspections,<sup>3</sup> involving users and clients as well as designers and developers, but recognize the effectiveness of heuristic

inspections and other techniques.

### **In practice**

The approach we took to designing a browser-resident classroom information management system illustrates the ad hoc accommodations an agile process often needs for development in Web time. A sophisticated performance-support system to empower K–12 classroom teachers, this system simplified key administrative, planning, and teaching tasks in an environment fully integrated with existing administrative systems and software and with Web-based learning resources. The application was deployed through a Web-based technology with the client-side user interface realized in HTML, XML, and Java JFC/Swing.

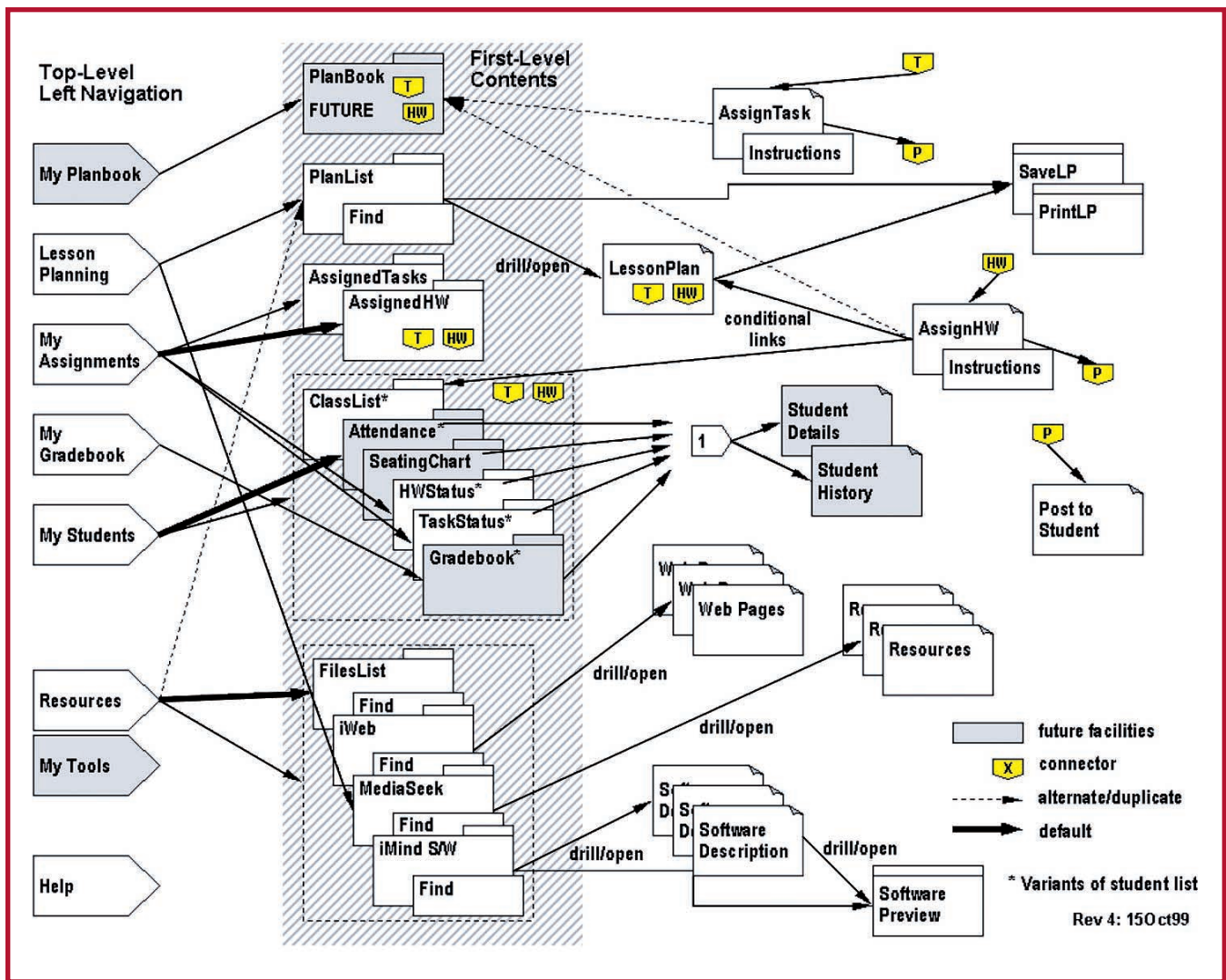
Risk factors were numerous, including a new and unproven development team in a new company, an outside consulting team for user interface design, the need for technical breakthroughs in several areas, vague and ambiguous requirements exacerbated by frequent changes in scope, and management obsessed with early delivery, frequent demonstrations, and unpredictable changes of direction.

The design team collaborated with a team of PhD-level educators with extensive classroom experience. This education team served as both domain experts and representative end users, while also functioning in what amounted to an XP-style customer role.<sup>2</sup>

Collaboratively with the education team, the design team developed the initial user role model and inventory of task cases. In total, we identified and prioritized 14 user roles. The initial inventory of 142 task cases included 35 task cases related to lesson planning, which emerged as a critical capability for the system. We wrote process narratives for only a handful of interesting task cases.<sup>1</sup> We constructed a draft navigation map with just over 40 interaction contexts, based on task case clustering (see Figure 5).

To resolve problems with the design and omissions, irregularities, and ambiguities in the requirements, we held frequent meetings and consulted continually with the education team—a pattern we now refer to as JITR, just-in-time requirements. One of the two designers concentrated on requirements and scope issues, while the other began working on the initial design.

**Each task cluster becomes the guide for designing a part of the user interface.**



**Figure 5. Web-deployed classroom information management system: Main portion of the navigation map.**

The design team devised drafts of a navigation architecture, navigation map, and visual and interaction scheme, validated them with the education team, then revised them.

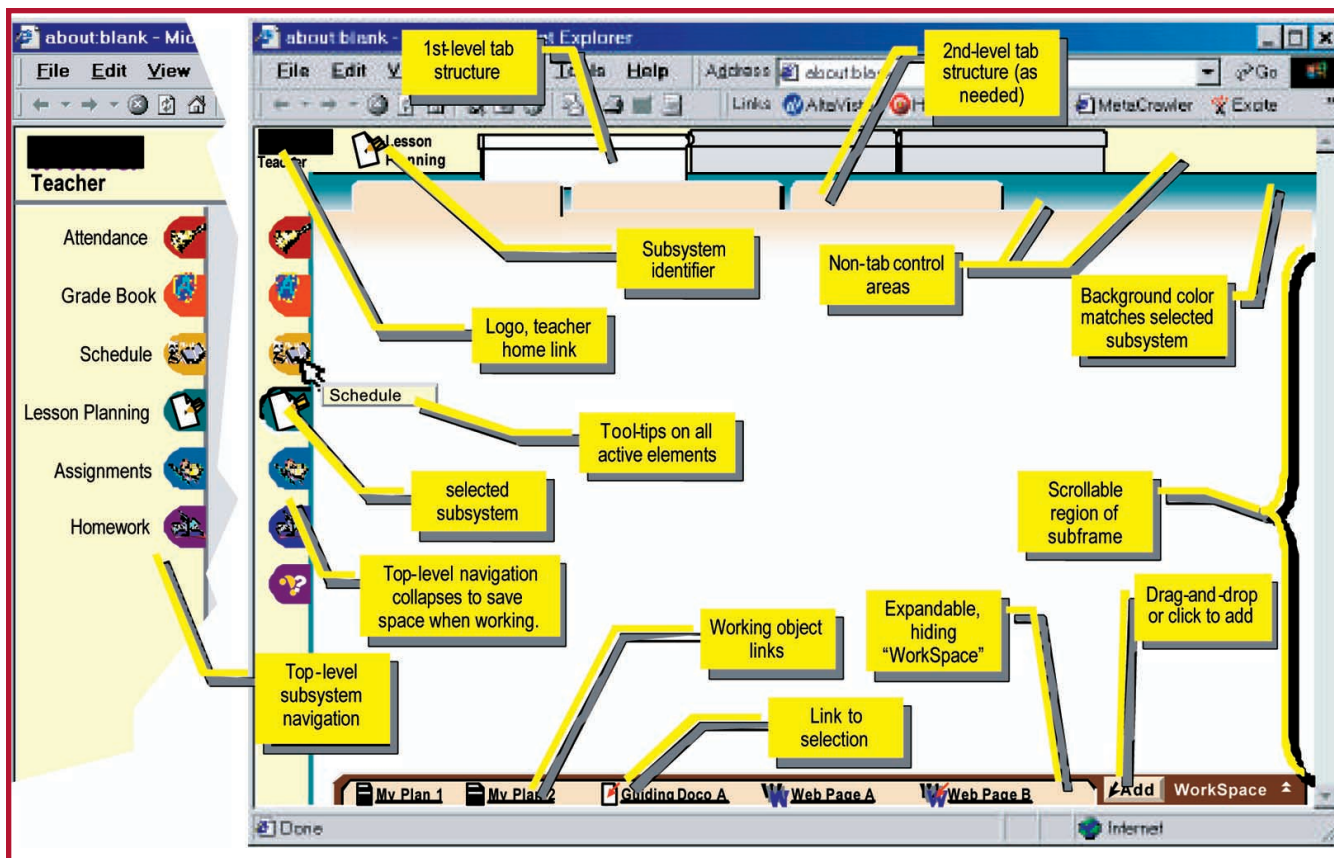
Not entirely by plan, the design proceeded in two- to two-and-a-half-week cycles. Once full-time design work started, we completed an initial navigation architecture and visual and interaction scheme within the first cycle. We finished the first version of the complete visual and interaction design in the next cycle, after which we inspected it for usability defects, and we undertook a major design revision on the second iteration. In the third iteration, we continued to extend and refine the design, spread over the remainder of the project's 17-week usage-centered design portion. Programming followed an agile, lightweight process that proceeded in parallel with visual and interaction design.

We needed a design that would be immediately usable by ordinary teachers

with little or no special training. It had to be flexible enough to accommodate a wide range of working styles and efficient to use, given that teachers may typically have less than 15 minutes a day to plan an entire day's lessons. Technical goals that we identified based on the usability criteria included

- Minimize window management overhead.
- Minimize modal behavior.
- Allow simple and direct switching among contexts.
- Maximize available screen real estate for document and data display.

A discussion of the complete design is beyond this article's scope (for details, see our design studies on the Web, [www.foruse.com/articles/designstudy1.pdf](http://www.foruse.com/articles/designstudy1.pdf), [/designstudy2.pdf](http://www.foruse.com/articles/designstudy2.pdf), and [/designstudy3.pdf](http://www.foruse.com/articles/designstudy3.pdf)). However, the first design iteration's results clearly illustrate the importance of early,



**Figure 6. Navigation architecture and design scheme, classroom information management system.**

up-front consideration of the overall navigation architecture and the visual and interaction scheme based on a full understanding of user roles and tasks. Neither traditional software engineering practices nor the newer lightweight methods make this possible.

Figure 5 shows the main portion of the navigation map, as finalized in the second iteration. The mock-up in Figure 6, adapted directly from actual design documents produced in the project's first cycle, illustrates how we documented the overall navigation and visual and interaction design scheme. Together, these design documents specified much of the user interface's overall architecture.

Understanding the various roles classroom teachers play and thoroughly exploring their tasks revealed the importance of a fast, flexible, and easy-to-learn scheme for navigating the application's various working contexts. Once we understood the task structure, we organized the various groups and interaction contexts for single-click switching among tasks within any given activity. Using simple visual metaphors based on thumb tabs, hanging folders, and file folders, we devised a three-level visible hierarchy that proved far easier to learn and

use than a conventional Windows-style tree view.

Screen real estate, always a scarce resource, is particularly valuable in an application running within a browser instance on low- to moderate-resolution displays. To address this, the design scheme included components that would expand and contract or reveal and hide themselves automatically as needed. For example, the top-level navigation panel (containing thumb tabs) contracts to provide more working space once the user selects a starting point. Figure 6 shows another example of screen space conservation in the dynamic workspace, which anticipated rather than mimicked the Apple OS-X dock. The workspace provides a compact holding bin that lets the user gather various documents or fragments from a variety of personal and public resources for assembly into a lesson plan or assignment.

Any product design's success is ultimately measured by its acceptance. Classroom teachers in a variety of settings made immediate use of this application with only minimal instruction. In fact, we were able to achieve the stringent design objective of enabling immediate, productive use of the system based on a single-page tutorial.

## Further Reading

- L. Constantine, *Constantine on Peopleware*, Prentice Hall, Upper Saddle River, N.J., 2001.
- R. Jeffries, A. Anderson, and C. Hendrickson, *Extreme Programming Installed*, Addison-Wesley, Reading, Mass., 2001.
- J. Nielsen and R.L. Mack, eds., *Usability Inspection Methods*, John Wiley & Sons, New York, 1994.

**O**ur experiences in agile usage-centered design have made clear that improved usability does not come without cost or risk, even when the methods are streamlined and the schedule is compressed. Special training and skills are necessary, and these may not be available to every development group, especially the lean teams typical in agile development. Moreover, the best design is easily corrupted by casual or even well-intentioned alterations and elaborations during coding. Close coordination between user interface designers and programmers is essential, and programmers must be fully committed to the process and convinced of the benefits even if they do not understand every design decision.

A flexible and condensed usage-centered design process is a good starting point for collaboration, but for agile practices such as Extreme Programming, which abjures anything resembling BDUF (big design up front), some of the rules and philosophy can get in the way. Just which rules need to be

bent or rewritten and what philosophy may need to be compromised is the subject of much current debate and experimentation. The outline presented here should be regarded more as a draft than as a definitive model. ☞

## References

1. P. Kruchten, *The Rational Unified Process: An Introduction*. Addison-Wesley, Reading, Mass., 1999.
2. K. Beck, *Extreme Programming Explained*, Addison-Wesley, Reading, Mass., 2000.
3. M. Fowler, "Put Your Processes on a Diet," *Software Development*, vol. 8, no. 12, Dec. 2000. Expanded version at [www.martinfowler.com/articles/newMethodology.html](http://www.martinfowler.com/articles/newMethodology.html).
4. L.L. Constantine and L.A.D. Lockwood, *Software for Use: A Practical Guide to the Models and Methods of Usage-Centered Design*, Addison-Wesley, Reading, Mass., 1999.
5. J. Anderson et al., "Integrating Usability Techniques into Software Development," *IEEE Software*, vol. 18, no. 1, Jan./Feb. 2001, pp. 46–53.
6. A. Parush, "Usability Design and Testing," *ACM Interactions*, vol. 8, no. 5, Sept./Oct. 2001.
7. I. Jacobson et al., *Object-Oriented Software Engineering: A Use Case Driven Approach*, Addison-Wesley, Reading, Mass., 1992.
8. S.M. McMenamin and J. Palmer, *Essential Systems Analysis*, Prentice Hall, Englewood Cliffs, N.J., 1984.
9. L.L. Constantine and L.A.D. Lockwood, "Structure and Style in Use Cases for User Interface Design." M. van Harmelan, ed., *Object Modeling an User Interface Design*, Addison Wesley, Reading, Mass., 2001.
10. D. Corlett, "Innovating with OVID," *ACM Interactions*, vol. 7, no. 4, July/Aug. 2000.
11. E. Gottesdiener, "Rules Rule: Business Rules as Requirements," *Software Development*, vol. 7, no. 12, Dec. 1999. Reprinted in L.L. Constantine, ed., *Beyond Chaos: The Expert Edge in Managing Software Development*, Addison-Wesley, Reading, Mass., 2001.
12. R. Jeffries, "Card Magic for Managers: Low-Tech Techniques for Design and Decisions," *Software Development*, vol. 8, no. 12, Dec. 2000. Reprinted in L.L. Constantine, ed., *Beyond Chaos: The Expert Edge in Managing Software Development*, Addison-Wesley, Reading, Mass., 2001.
13. L.A.D. Lockwood, "Taming the Wild Web: Business Alignment in Web Development," *Software Development*, vol. 7, no. 4, Apr. 1999. Reprinted in L.L. Constantine, ed., *Beyond Chaos: The Expert Edge in Managing Software Development*, Addison-Wesley, Boston, 2001.
14. M. Fowler, *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, Reading, Mass., 1999.
15. M.H. Cloyd, "Designing User-Centered Web Applications in Web Time," *IEEE Software*, vol. 18, no. 1, Jan./Feb. 2001, pp. 62–69.
16. J. Pokorny, "Static Pages Are Dead: How a Modular Approach Is Changing Interaction Design," *ACM Interactions*, vol. 8, no. 5, Sept./Oct. 2001.

## About the Authors



**Larry L. Constantine** is an adjunct professor of information technology, University of Technology, Sydney, Australia, and director of research, Constantine & Lockwood, Ltd. A methodologist who pioneered fundamental concepts and techniques underlying modern software engineering theory and practice, he now focuses on methods for improving the usability of software and hardware. A consultant and designer with clients around the world, he shared the 2001 Performance-Centered Design award for his work with Siemens AG. He is a graduate of MIT's Sloan School of Management, a member of the IEEE Computer Society and the ACM, and he has served on the boards of numerous professional journals, including *IEEE Software*. Contact him at [lconstantine@foruse.com](mailto:lconstantine@foruse.com).

**Lucy A.D. Lockwood** is president of Constantine & Lockwood, Ltd., the design, training, and consulting firm she cofounded. One of the developers of usage-centered design, she is the coauthor of a Jolt Award-winning book on the subject. With a passion for teaching and professional communication, she has chaired several professional conferences and is co-organizer of forUSE 2002, the first international conference on usage-centered, performance-centered, and task-oriented design. She is a graduate of Tufts University and a member of the IEEE and the ACM. Contact her at [llockwood@foruse.com](mailto:llockwood@foruse.com).



For more information on this or any other computing topic, please visit our Digital Library at <http://computer.org/publications/dlib>.